

GeekOnSites (GOS)

Full Project Overview & System Architecture

Project Purpose

GeekOnSites is a US/UK-focused technology support and service-booking platform that allows customers to discover IT services, request/book support, make secure payments, receive technician assignment, track service progress, and complete remote or on-site support. The platform is designed as a premium, mobile-first SaaS product with separate operational portals for customers, technicians, agents, and administrators.

1. What GeekOnSites Provides

- IT and technical support services for customers in the US and UK.
- Remote support and on-site technician services.
- Service discovery with transparent US dollar / UK pound pricing.
- Online booking, secure Stripe payment, booking confirmation and service tracking.
- Technician assignment and location tracking for applicable on-site services.
- Remote support through a shared Google Meet session.
- Customer notifications, booking history, invoices and service ratings.
- Operational management through Technician, Agent and Admin portals.

2. Main User Roles

Role	Main Responsibilities
Customer	Browse services, book, pay, view bookings, receive notifications, track technician, join remote session, rate s
Technician	Register/verify, manage availability, receive assignments, update service status, communicate/session supp
Agent	Support customers, manage operational requests, coordinate bookings and technician-related activities.
Admin	Manage users, services, bookings, technicians, agents, customer messages, payments and overall platform operations.

3. Complete Customer Journey

Home → Services → Select Service → Book Service → Payment → Booking Success → Technician Assigned → Tracking / Remote Session → Service Completed → Invoice → Rating & Review

The booking lifecycle is controlled by the backend so that payment, assignment, service progress and completion remain synchronized across customer and operational portals.

4. Technician Workflow

- Technician registers and completes required verification.
- Technician becomes available for eligible service assignments.
- System/operations assigns a technician to a confirmed booking.
- Technician views customer/service details and updates job status.
- For remote work, technician uses the shared meeting link.
- For on-site work, technician can use location/tracking functionality.
- Technician completes the service and updates the booking.
- Customer receives completion information and can provide a rating/review.

5. Agent & Admin Operations

Agent: Handles customer support, booking coordination, operational follow-up and service-related assistance.

Admin: Has centralized control over customers, technicians, agents, services, bookings, payments, notifications, contact messages and platform monitoring.

6. Technology Stack

Layer	Technology / Platform
Frontend	React + Vite + Tailwind CSS + Framer Motion
Backend	Java + Spring Boot REST API
Authentication	JWT + role-based authorization
Database	MySQL — Railway
Payments	Stripe Checkout + Stripe Webhooks
Remote Sessions	Google Meet
Hosting	Frontend: Hostinger Backend: Render
Location	Maps/location integration for technician tracking

7. System Architecture

Customer / Technician / Agent / Admin

↓

React Web Application

↓ HTTPS / REST API

Spring Boot Backend

- Authentication & Authorization
- Customer Management
- Service Management
- Booking Management
- Technician Management
- Agent & Admin Management
- Payment Management
- Notification Management
- Remote Session / Tracking Logic

↓

Railway MySQL Database

8. Key Backend Entities

Customer • Booking • Technician • Agent • Admin/User • Service • Payment • Notification • ContactMessage • Session/Meeting • Rating/Review

9. Payment Architecture

Customer selects a service → backend creates a Stripe Checkout session → customer completes payment on Stripe → Stripe webhook confirms the payment → backend updates payment/booking status → customer receives booking confirmation. Payment status is validated server-side rather than relying only on frontend information.

10. Frontend Structure

Public: Home, Services, About, Contact

Customer: Dashboard, Book Service, Payment, My Bookings, Notifications, Profile, Tracking, Remote Session, Invoice, Rating

Technician: Login/Register, Verification, Dashboard, Assignments, Service Status, Remote Session

Agent: Login, Dashboard, Customer/Booking Operations

Admin: Login, Dashboard, Users, Services, Bookings, Technicians, Agents, Messages, Payments and monitoring

11. Navigation Design

Desktop: Home | Services | About | Contact | Portal | Agent

Customer Mobile: Home | Services | Bookings | Notify | Profile

The same service catalog is designed to support both the desktop website and mobile experience.

12. Security & Production Controls

- HTTPS communication between frontend and backend.
- JWT authentication and role-based access control.
- Passwords and secrets kept outside source code using environment variables.

- Server-side validation for bookings and payments.
- Stripe webhook verification.
- Database constraints and API validation.
- Production error handling and logging.
- No hardcoded customer, booking, payment or technician IDs.

13. Deployment Architecture

Frontend: React production build hosted on Hostinger and served through the GeekOnSites domain. **Backend:** Spring Boot application deployed on Render.

Database: Cloud MySQL on Railway.

External services: Stripe for payments, Google Meet for remote sessions, and Maps/location services for tracking.

14. Business & Pricing Principles

- US and UK markets only; no India customer pricing or currency in the production customer flow.
- Pricing displayed in USD for US customers and GBP for UK customers.
- Platform/service fees can be applied according to the defined pricing policy.
- Free quote where applicable.
- No-fix/no-fee policy where applicable.
- Hardware/parts billed separately when required.
- Remote support available by appointment, with emergency service pricing where applicable.

15. Future Scalability

The architecture is designed so additional services, technicians, automated technician matching, richer notifications, mobile applications, advanced analytics, automated scheduling and additional payment/communication integrations can be added without replacing the core platform.

16. Project Objective

Build GeekOnSites as a reliable, premium and scalable US/UK technical-services marketplace where the complete journey—from service discovery and payment to technician delivery and post-service support—is managed through one connected platform.

Manager Summary: GeekOnSites combines a customer-facing service marketplace with operational portals and a secure backend. React provides the user experience, Spring Boot manages business logic and APIs, MySQL stores platform data, Stripe handles payments, and external services support remote sessions and technician tracking.